ChartNav — Consolidated Build

Backend phases 1–5: spine → state machine → auth → RBAC → CI/hardening
Repo: `thekidd2227/chartnav-platform`

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. 09 — CI & deploy hardening
11. 10 — Doc artifact pipeline
12. 11 — Production-auth seam
13. 12 — Runtime config
14. 13 — Deploy target
15. 14 — Postgres parity
16. 15 — Frontend integration
17. Diagram — System architecture
18. Diagram — Encounter status machine
19. Diagram — ER
20. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Six encounter endpoints. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. `apps/api/app/auth.py` resolves the caller from `X-User-Email`. Every encounter route became org-scoped: cross-org reads return 404, cross-org body/query assertions 403.

Phase 4 — RBAC + full scoping + tests. `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and scoped.

`{error_code, reason}` envelope is standardized. 25-test pytest suite covers auth, scoping, RBAC, state-machine invariants.

Phase 5 — CI + runtime hardening. GitHub Actions workflow runs install + Alembic upgrade on an isolated CI DB + idempotent seed + pytest + live smoke. A separate `docs` job regenerates the final HTML/PDF. Root `Makefile`, `apps/api/scripts/smoke.sh`, and `scripts/build_docs.py` codify the local verification + doc-build paths.

Phase 6 — Production seam + deploy target + Postgres parity. DB layer moved to SQLAlchemy Core. `apps/api/app/config.py` centralizes env. `auth.py` gained a bearer stub that returns 501 honestly. Dockerfile hardened, `docker-compose.prod.yml` + `scripts/pg_verify.sh` + `backend-postgres` CI job land.

Phase 7 — Frontend workflow UI (this phase). `apps/web/` becomes a real app. Typed API client (`src/api.ts`) funnels every call through one surface that converts backend 4xx envelopes into `ApiError` for the UI. `src/identity.ts` persists the seeded dev caller in `localStorage`. `App.tsx` renders a two-pane workflow console: filtered encounter list, encounter detail with a facts grid and color-coded status, an event timeline, role-aware transition buttons, and a composer for adding events (hidden with explanation for reviewers). Error banners surface the exact `{error_code, reason}` the backend ships. `make dev` boots API + Vite together with trap-based teardown. No backend contract changes.

Preserved untouched: `/health`, `/`, Alembic history, SQLite dev workflow, state machine + filtering surface, workflow_events model, existing endpoint contracts.

01 — Current state

ChartNav — Current State

As of: 2026-04-18 (phase: frontend workflow UI)

Repo layout (relevant)

```
chartnav-platform/
├── .github/workflows/ci.yml     # backend-sqlite · backend-postgres · docker-build · docs
├── Makefile                     # install · verify · pg-verify · docker-* · web-* · dev
├── scripts/                     # build_docs.py · verify.sh · pg_verify.sh
├── apps/
│   ├── api/
│   │   ├── Dockerfile · entrypoint.sh
│   │   ├── .env.example
│   │   ├── app/{main,config,db,auth,authz}.py
│   │   ├── app/api/routes.py
│   │   ├── alembic/
│   │   ├── scripts_seed.py
│   │   ├── scripts/smoke.sh
│   │   └── tests/               # 28 pytest incl. 3 auth-mode tests
│   └── web/
│       ├── .env.example         # NEW — VITE_API_URL contract
│       ├── src/
│       │   ├── api.ts           # NEW — typed API client
│       │   ├── identity.ts      # NEW — dev identity helpers
│       │   ├── App.tsx          # NEW — full workflow UI
│       │   ├── main.tsx         # wires App.tsx + styles.css
│       │   ├── styles.css       # NEW — single CSS file
│       │   └── vite-env.d.ts    # NEW — Vite ambient types
│       └── package.json
├── infra/docker/{docker-compose.yml,docker-compose.prod.yml}
└── docs/build/01..15            # now including 15-frontend-integration.md
```

Runtime baseline

- Backend: Python 3.11, FastAPI, SQLAlchemy Core, SQLite or Postgres.
- Frontend: Vite 5 + React 18 + TypeScript; vanilla CSS.
- Auth: `CHARTNAV_AUTH_MODE=header` (dev) or `bearer` (placeholder 501).
- DB: SQLite default (`apps/api/chartnav.db`), Postgres via `DATABASE_URL=postgresql+psycopg://…` .
- Alembic head: `a1b2c3d4e5f6` . No schema changes this phase.
- RBAC: `admin` , `clinician` , `reviewer` .
- Error envelope: `{"detail": {"error_code": "...", "reason": "..."}}` — the frontend surfaces these verbatim.

Frontend capabilities (new)

- Header with brand, caller chip (`email · role · org N`), API base URL, identity picker (5 seeded users + custom).
- Encounter list with filters (`status` , `provider_name` , `location_id`) and color-coded status pills.
- Encounter detail with facts grid, current status, allowed transitions (role-aware), event timeline.
- Event composer for admin / clinician; hidden with explanation for reviewer.
- All API errors surface as banners with `error_code` + `reason` .
- Dev identity persists in `localStorage` ; switching reloads `/me` and list.

Verified working endpoints

No endpoint behavior changed this phase. All 28 pytest tests still pass;

the UI exercises the full surface through the typed client.

Automation

- `make verify` — SQLite: reset-db + pytest + boot + smoke.
- `make pg-verify` — Postgres parity proof.
- `make web-install` / `web-dev` / `web-build` / `web-typecheck` .
- `make dev` — boots API (8000) + Vite (5173) together with a shared trap-based teardown.
- CI: `backend-sqlite` → `backend-postgres` + `docker-build` + `docs` .

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)
- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next states from in_
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.
- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]` , one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000` . All error bodies use

`{"detail": {"error_code": "...", "reason": "..."}}` .

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped)

Method	Path	Behavior
GET	`/organizations`	Returns only caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org` .
GET	`/users`	`WHERE organization_id = caller.org` .

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always `WHERE organization_id = caller.org` .

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.

Param	Rule
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403. 404 if cross-org.

POST `/encounters/{id}/status`

Per-edge RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`. Cross-org → 404.

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles`
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts other org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters bad init
`invalid_transition`	400	disallowed state-machine edge

Verification

This full surface is now locked in by two layers:

- pytest (`apps/api/tests/`) — 25 tests, per-test ephemeral SQLite.
- Live smoke (`apps/api/scripts/smoke.sh`) — 9 assertions against a running API.

Both run in CI on every push/PR. See `09-ci-and-deploy-hardening.md` .

04 — Data model

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6` . No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL
role	VARCHAR	NOT NULL default `"admin"`
created_at	DATETIME	NOT NULL default now()

Application-level role vocabulary (enforced by `app/authz.py`):

`admin` , `clinician` , `reviewer` . The column remains a free VARCHAR in

SQL; constraint is enforced at the app layer. If you need a DB-level

CHECK constraint, add it in a future migration.

`encounters` and `workflow_events`

Unchanged.

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md`.

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological.

2026-04-18 — Phase 7: frontend workflow UI

Step 1 — Baseline

- Starting head: `700bb0b` (prod auth seam + Docker + Postgres parity).
- `apps/web/` was a placeholder — a single `App.jsx` hitting `/health` and an unrelated `main.tsx`.

Step 2 — Typed API client

- New `apps/web/src/api.ts`:
- `API_URL` from `VITE_API_URL`, fallback `http://localhost:8000`.
- `ApiError(status, error_code, reason)` surfaces the backend envelope.
- Typed wrappers for `/health`, `/me`, `/encounters`, `/encounters/{id}`, `/encounters/{id}/events`, `POST /encounters/{id}/events`, `POST /encounters/{id}/status`, `/locations`.
- Pure helpers `allowedNextStatuses(role, status)` and `canCreateEvent(role)` that mirror `authz.TRANSITION_ROLES` — for UI affordances only; backend still governs.
- New `apps/web/src/identity.ts` — seeded users list (all 5 demo identities), localStorage persistence, safe fallback.
- New `apps/web/src/vite-env.d.ts` — Vite ambient types for `import.meta.env`.

Step 3 — App shell

- Rewrote `apps/web/src/App.tsx` as the full workflow app:
- Sticky header: brand, caller chip, API chip, identity picker (seeded + custom).
- Two-column layout (collapses on mobile).
- `apps/web/src/main.tsx` imports `App` + `styles.css`.
- New `apps/web/src/styles.css`: one file, CSS variables, color-coded status pills, filter/list/detail/timeline/form styles.

Step 4 — Encounter list + filters

- Filter bar for `status`, `provider_name`, `location_id`; `clear` button when filters are set.
- Loading / empty / error states in the list.
- Row click selects an encounter (and keyboard Enter/Space).

Step 5 — Encounter detail + timeline

- Facts grid (org, location, scheduled/started/completed/created).

- Color-coded status pill.
- Timeline renders `event_data` as a readable `key: value · …` line (objects), verbatim for strings; falls back to `–` when null.

Step 6 — Role-aware actions

- Allowed transitions rendered as buttons based on `(role, current_status)`. When none are allowed, a clean note explains why — no fake-disabled buttons.
- Event composer hidden for reviewers with an explanation; admins + clinicians get it.
- On success: refresh detail + events + list; show a green banner.
- On failure: render the exact `{status} {error_code} – {reason}` string in a red banner. Nothing is silently swallowed.

Step 7 — Dev UX

- New `apps/web/.env.example` — `VITE_API_URL`.
- Makefile: `web-install`, `web-dev`, `web-build`, `web-typecheck`, `dev` (boots both).
- `make dev` uses a trap-based teardown so Ctrl-C kills both backend and frontend.

Step 8 — Local verification

- `npx tsc --noEmit` clean after adding `vite-env.d.ts`.
- `npm run build` → `dist/` produced (154 KB JS / 6 KB CSS / 0.4 KB HTML).
- Ran uvicorn + exercised every endpoint the UI depends on against all 5 seeded roles:
- `/me` returns 200 for all 5, 401 for unknown/empty.
- `/encounters` scoped correctly (org1 sees 2 rows, org2 sees `[3]`).
- Filter `status=in_progress` → `['PT-1001']`.
- Clinician `in_progress → draft_ready` → 200.
- Clinician `review_needed → completed` → 403 `role_cannot_transition`.
- Reviewer `review_needed → completed` → 200.
- Reviewer `POST event` → 403 `role_cannot_create_event`.
- Admin `POST event` → 201.
- Backend tests still pass (no backend changes this phase).

Step 9 — Docs

- New: `15-frontend-integration.md`.
- Updated: `01-current-state.md`, `05-build-log.md`, `06-known-gaps.md`, `08-test-strategy.md`, `12-runtime-config.md`.
- Diagrams updated: `system-architecture`, `api-data-flow` (with frontend layer).
- `scripts/build_docs.py` now picks up section 15.
- Final HTML + PDF regenerated.

Step 10 — Hygiene

- Removed stale `apps/web/src/App.jsx`.
- `.gitignore` already excludes `node_modules`, `dist`, caches; reran `npm install` and only `package-lock.json` grew.

- Dev DB reset before commit.

Prior phases

- **Phase 6 — Prod auth seam + Docker + Postgres parity** (`700bb0b`)
- **Phase 5 — CI + runtime hardening + doc pipeline** (`cfa8ca9`)
- **Phase 4 — RBAC + full scoping + pytest** (`c6f29e6`)
- **Phase 3 — Dev auth + org scoping** (`efb5b56`)
- **Phase 2 — Strict state machine + filtering** (`505f025`)
- **Phase 1 — Workflow spine** (`93fceb4`)

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Verification evidence — phase 7 (frontend)

Backend still green

- `make verify` (SQLite): **28/28 pytest** + 9/9 smoke + clean teardown.
- No backend code changed this phase. All previous verification stands.

Frontend build + typecheck

```
$ cd apps/web && npx tsc --noEmit   # clean
$ npm run build
✓ 33 modules transformed.
dist/index.html                   0.40 kB │ gzip:  0.27 kB
dist/assets/index-*.css           6.24 kB │ gzip:  1.79 kB
dist/assets/index-*.js          154.44 kB │ gzip: 49.66 kB
✓ built in 639ms
```

Live integration (uvicorn + curl as a UI stand-in)

Flow the UI depends on	Result
`GET /me` for all 5 seeded identities	✅ 200
`GET /me` unknown / empty	✅ 401 (UI shows red chip)
`GET /encounters` org1 vs org2 scoping	✅ disjoint
`GET /encounters?status=in_progress` filter	✅ `['PT-1001']`
`GET /encounters/{id}` + `GET /encounters/{id}/events`	✅ 200, 3 events
Clinician `in_progress → draft_ready`	✅ 200
Clinician `review_needed → completed`	✅ 403 `role_cannot_transition`
Reviewer `review_needed → completed`	✅ 200
Reviewer `POST /encounters/{id}/events`	✅ 403 `role_cannot_create_event`
Admin `POST /encounters/{id}/events`	✅ 201

UI affordances sanity

Role	Transition buttons shown	Event composer
admin	Any forward or rework edge valid from the current state	visible
clinician	`scheduled→in_progress`, `in_progress→draft_ready`, rework back	visible
reviewer	`draft_ready→review_needed`, `review_needed→{completed,draft_ready}`	hidden w/ note

When no transition is available (terminal `completed` or role mismatch),

the UI shows a note rather than fake-disabled buttons. Backend remains

authoritative — a mismatch between the UI's `allowedNextStatuses` and

the server is surfaced as a 4xx banner with the exact `error_code`.

Real gaps (prioritized for next phase)

1. **No automated frontend tests.** All frontend verification this phase is build/typecheck + live-curl parity. Next: a small Vitest or Playwright pass.

2. **JWT validation still stubbed.** Bearer mode returns 501; the frontend currently only implements header mode.

3. **No encounter creation UI.** The frontend can add events and move statuses but cannot create encounters yet (backend endpoint exists).

4. **No pagination.** Works fine at seed scale; first real dataset will need it on both ends.

5. **No optimistic updates.** Every mutation re-fetches.

6. **No global state manager.** Fine at this size; watch for prop-drill as flows multiply.

7. **No encounter update / delete / cancel.** No `cancelled` status.

8. **Free-form `event_data`** still lacks per-event_type schema.

9. **Raw `sqlite3` → SA Core refactor complete** — pytest matrix against Postgres is the next CI upgrade.

10. **CORS `allow_origins=["*"]` ** remains.

11. **No distinct audit log** for auth/scoping failures.

12. **No rate limiting, lockout, or structured logging.**

13. **No CI job for the frontend.** The `docker-build` + `docs` jobs don't cover the web bundle. A `frontend-build` CI job is a small, honest addition.

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.

Code	HTTP	When
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. What this phase explicitly does NOT do

- No password auth, JWT, or SSO integration.
- No scoped writes to organizations / locations / users (meta admin).
- No audit log distinct from `workflow_events`.
- No rate limiting.
- No per-tenant key management.

These are the next auth/security phase's targets.

Test & Smoke Strategy

Stack

- `pytest` + FastAPI `TestClient` for unit/integration.
- Per-test temporary SQLite (fresh migrate + seed per test); env-based wiring so the same fixture will later work against Postgres.
- Live shell smoke (`apps/api/scripts/smoke.sh`) used by:
- `make verify` (SQLite)
- `scripts/pg_verify.sh` (Postgres)
- the built Docker image smoke job
- CI runs all of the above on every push / PR.

Layout

```
apps/api/tests/
├── conftest.py          # env-based per-test DB, header presets
├── test_auth.py         # authn surface (5)
├── test_auth_modes.py   # header + bearer seam (3)
├── test_rbac.py         # role-gated writes + state transitions (12)
└── test_scoping.py      # org scoping (8)

apps/api/scripts/
└── smoke.sh             # curl-level live smoke (9 checks)

scripts/
├── verify.sh            # boot + smoke + teardown (SQLite)
└── pg_verify.sh         # same flow against a throwaway Postgres
```

Coverage

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
`/health` , `/`	✓	✓	✓	✓
Header-mode auth	✓	✓	✓	✓
Bearer-mode seam (501 / config)	✓			
Org scoping	✓	✓	✓	✓
RBAC	✓			
Filter on `/encounters`	✓			

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
State machine invariants	✓		✓	
Event provenance (`changed_by`)	✓		✓	
Migrations			✓	✓
Seed idempotency (x2 runs)			✓	

How to run

```
# Fast pytest (SQLite)
cd apps/api && pytest tests/ -v

# Full local SQLite gate
make verify                 # resets DB, pytest, boot, smoke, teardown

# Postgres parity (docker required)
make pg-verify

# Docker image + smoke
make docker-build
```

Results at time of writing

- `pytest` : **28 passed in ~19s**
- `make verify` (SQLite): 28 pytest + 9 smoke — all green
- `scripts/pg_verify.sh` : migrations + seed (x2) + smoke + status transition + event write — **PASS**

Frontend verification (phase 7)

- `npx tsc --noEmit` — clean.
- `npm run build` — emits `dist/index.html` + hashed JS/CSS.
- Live integration against uvicorn: every endpoint the UI hits

returns the expected status across all 5 seeded roles (see

`06-known-gaps.md` matrix).

- No automated frontend tests yet — next item.

Gaps not yet covered

- pytest matrix against Postgres (the fixture is already env-driven).
- Concurrent writers / race conditions.
- `event_data` schema per `event_type` .
- Pagination behavior (not yet implemented).

- Bearer-mode JWT validation (placeholder only; see `11-production-auth-seam.md`).

CI & Deploy Hardening

GitHub Actions

File: `.github/workflows/ci.yml`. Triggers on **push to `main`** and **every pull request**.

`backend-sqlite`

Install deps → `alembic upgrade head` against an isolated `$RUNNER_TEMP/chartnav_ci.db` via `DATABASE_URL` → seed **twice** (idempotency proof) → `pytest tests/ -v` (28 tests) → `bash scripts/verify.sh` (boots uvicorn, runs `scripts/smoke.sh`).

`backend-postgres` (needs: `backend-sqlite`)

Service container `postgres:16-alpine` on localhost:5432. Install deps (incl. `[postgres]` extra) → `alembic upgrade head` on Postgres → seed twice → boot uvicorn → `scripts/smoke.sh` → live status transition asserts the returned row. Fails the workflow if Postgres parity breaks.

`docker-build` (needs: `backend-sqlite`)

Buildx → build `chartnav-api:ci` from `apps/api/` → run the container with `DATABASE_URL=sqlite:///./chartnav.db` and `CHARTNAV_RUN_SEED=1` → poll `/health` → run `scripts/smoke.sh` against the live container. Proves the production image boots end-to-end.

`docs` (needs: `backend-sqlite`)

apt-install Chromium → `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser` → upload HTML + PDF as `chartnav-docs-final` artifact (`if-no-files-found: error`).

What CI catches

Change	Failure surface
Broken migration	alembic step (both DBs)
Broken seed / re-seed collision	seed step (both DBs)
Any pytest regression	`backend-sqlite` pytest
`/health` or auth drift	`scripts/smoke.sh` (all envs)
Cross-org / state-machine regression	smoke + `backend-postgres` transition
Postgres-only SQL dialect break	`backend-postgres`
Image won't build or boot	`docker-build`
Docs pipeline regression	`docs` job

Local verification

```
make verify        # SQLite end-to-end
make pg-verify     # Postgres end-to-end (Docker required)
make docker-build  # build the production image
make docker-up     # full API + Postgres stack (compose)
```

Individual recipes: `install	migrate	seed	test	boot	smoke	docs	reset-db	clean`.

Dev / test / CI / prod DB separation

Surface	Path
Developer local	`apps/api/chartnav.db` (gitignored)
pytest per-test	`tmp_path / "chartnav.db"` (ephemeral)
CI (sqlite)	`$RUNNER_TEMP/chartnav_ci.db` (ephemeral)
CI (postgres)	`postgres:16-alpine` service container
Local pg-verify	throwaway `chartnav-pg-verify` container on port 55432
Prod compose	`chartnav_pgdata` persistent volume

Smoke script (shared contract)

`apps/api/scripts/smoke.sh [BASE_URL]` — 9 curl assertions:

`/health` (code + body), `/me` (401/200), `/encounters` (401/200), cross-org lens 403, own 200, cross-org 404. Exits non-zero on first failure. Used by **every** smoke path (local `make verify`, Postgres parity, Docker image CI).

What this phase adds beyond CI

- Hardened `Dockerfile` (non-root, healthcheck).
- `apps/api/entrypoint.sh` for deterministic migrate-on-start.
- `infra/docker/docker-compose.prod.yml` for the API + Postgres stack.
- `scripts/pg_verify.sh` — reproducible Postgres parity proof.

What this phase does NOT do

- No image push / registry release.
- No Kubernetes / Terraform / cloud-specific bits.
- No TLS termination or edge reverse proxy.
- No secret store integration.

Those land in a dedicated hosted-deploy phase. This phase makes sure

the seams are in place to plug any of them in without rewrites.

10 — Doc artifact pipeline

Doc Artifact Pipeline

Purpose

The living docs in `docs/build/*` and `docs/diagrams/*` are the source

of truth. The final consolidated artifacts in `docs/final/` are always

rebuildable from those sources — no hand edits.

Source / output layout

```
docs/
├── build/        # human-maintained markdown — source of truth
├── diagrams/     # Mermaid sources (fenced in markdown)
└── final/
    ├── chartnav-workflow-state-machine-build.html   # generated
    └── chartnav-workflow-state-machine-build.pdf    # generated
```

Builder

File: `scripts/build_docs.py`

- Pure Python, no external deps beyond the standard library.
- Resolves paths relative to the script, so the same invocation works

locally and in CI.

- Parses a small markdown subset (headings, fenced code, tables, bullet

lists, inline `code`, **`bold`**) directly to HTML.

- Mermaid fences are passed through verbatim into `<div class="mermaid">`

blocks; the HTML imports Mermaid from a CDN and renders on load.

- Writes `docs/final/chartnav-workflow-state-machine-build.html`.
- Generates the PDF via headless Chromium (`--headless=new --print-to-pdf`).

Selection order:

1. `CHARTNAV_PDF_BROWSER` env var if set (used by CI with `chromium-browser`).

2. macOS Chrome at `/Applications/Google Chrome.app/...`.

3. Chromium in a standard app bundle.

4. `chromium-browser`, `chromium`, `google-chrome`, `chrome` on `PATH`.

- If no browser is available, falls back to `reportlab` for a plain-text

PDF so CI never silently produces a zero-byte artifact.

Run it

Locally:

```
python scripts/build_docs.py          # or:  make docs
```

In CI: the `docs` job in `.github/workflows/ci.yml` installs

`chromium-browser` via apt, runs the builder with

`CHARTNAV_PDF_BROWSER=chromium-browser`, and uploads the resulting

HTML + PDF as a `chartnav-docs-final` artifact. The upload step is

configured with `if-no-files-found: error`, so a silent generation

failure fails the build.

Sections consumed (in order)

1. Executive summary (inlined in the builder)
2. `docs/build/01-current-state.md`
3. `docs/build/02-workflow-state-machine.md`
4. `docs/build/03-api-endpoints.md`
5. `docs/build/04-data-model.md`
6. `docs/build/05-build-log.md`
7. `docs/build/06-known-gaps.md`
8. `docs/build/07-auth-and-scoping.md`
9. `docs/build/08-test-strategy.md`
10. `docs/build/09-ci-and-deploy-hardening.md`
11. `docs/build/10-doc-artifact-pipeline.md`
12. `docs/diagrams/system-architecture.md`
13. `docs/diagrams/encounter-status-machine.md`
14. `docs/diagrams/er-diagram.md`
15. `docs/diagrams/api-data-flow.md`

If a source file is missing, the builder emits an inline `*missing: <path>*` marker rather than crashing — a broken diagram or an undeleted section shows up immediately in the rendered output and in the CI artifact.

Why not a heavier stack

- No MkDocs / Docusaurus / Sphinx — those require a theme + build config we'd have to maintain. We currently just need the 10-page PDF to stay fresh.
- No pandoc dependency — keeps the CI image lean and the local experience `python` + a headless browser (which most devs already have).
- If we grow beyond a single consolidated artifact (e.g. a hosted docs site), this builder is easy to retire because the sources are already plain markdown.

11 — Production-auth seam

Production-Auth Seam

Goal

Make auth transport swappable without touching routes or RBAC.

Where the seam lives

`apps/api/app/auth.py` exposes a single FastAPI dependency:

```
def require_caller(x_user_email: ..., authorization: ...) -> Caller:
    mode = settings.auth_mode
    if mode == "header":  return resolve_caller_from_header(...)
    if mode == "bearer":  return resolve_caller_from_bearer(...)
    raise ...
```

Both resolvers return the **same** `Caller` contract:

```
Caller(user_id, email, full_name, role, organization_id)
```

Every route, every RBAC helper, every test depends on `Caller`, not on a

specific header. Changing transports is a one-file edit inside `auth.py`.

Mode selection

`CHARTNAV_AUTH_MODE` in the environment (see `12-runtime-config.md`):

Value	State	Behavior
`header`	implemented	Reads `X-User-Email`, looks up `users`. Dev only.
`bearer`	placeholder	Requires `Authorization: Bearer …`; returns 501 because JWT validation is not yet wired.

Unknown values cause `app.config` to refuse to import — fail fast.

Bearer mode — honest placeholder

When `CHARTNAV_AUTH_MODE=bearer` is selected, `app.config._load()`

verifies these three values are all present before letting the app come

up at all:

- `CHARTNAV_JWT_ISSUER`
- `CHARTNAV_JWT_AUDIENCE`
- `CHARTNAV_JWT_JWKS_URL`

Missing any one → `RuntimeError` at import time with the list of missing vars. The app will not start with a half-configured bearer setup.

When bearer mode is active and fully configured, `resolve_caller_from_bearer` currently returns:

```
HTTP 501
{"detail": {"error_code": "auth_bearer_not_implemented",
            "reason": "bearer-token validation is not implemented in this phase; set CHARTNAV_AUTH_MOD
```

This is intentional. A future phase will wire real JWT signature verification (PyJWT + JWKS fetch against `CHARTNAV_JWT_JWKS_URL`, issuer/audience checks, claim → `users` lookup). When it ships, only the body of `resolve_caller_from_bearer` changes.

What production-ready vs not yet implemented

Component	State
Mode selection + config	✅ production-shaped; fails fast on gaps
Standardized error envelope	✅ `{error_code, reason}` everywhere
Header transport	✅ works; dev only, do not expose
Bearer transport	⚠ placeholder — returns 501 honestly
JWT signature validation	❌ not implemented
JWKS rotation / cache	❌ not implemented
Token → `users` mapping	❌ not implemented (design TBD)
RBAC integration	✅ works for any transport via `Caller`
Org scoping integration	✅ works for any transport via `Caller`

Tests

`apps/api/tests/test_auth_modes.py`:

- `test_bearer_mode_requires_jwt_env` — setting `CHARTNAV_AUTH_MODE=bearer`

without issuer/audience/JWKS raises at `import app.config`.

- `test_bearer_mode_refuses_to_serve_traffic` — with JWT config set, bearer requests still get a clear 501 `auth_bearer_not_implemented`; missing header returns 401 `missing_auth_header`.
- `test_header_mode_unchanged_contract` — header mode (the default) still resolves seeded users to the expected role/org.

12 — Runtime config

Runtime Configuration

Single source of truth: `apps/api/app/config.py` (`settings`). Every

other module imports from there; nothing else reads `os.environ` .

The contract

Variable	Required?	Default	Purpose
`CHARTNAV_ENV`	no	`dev`	Informational label (`dev` / `test` / `ci` / `prod`).
`DATABASE_URL`	no (dev default)	`sqlite:///apps/api/chartnav.db`	SQLAlchemy URL. Supports `sqlite:///...` and `postgresql+psycopg://...` .
`CHARTNAV_AUTH_MODE`	no	`header`	`header` (dev only) or `bearer` (prod placeholder).
`CHARTNAV_JWT_ISSUER`	**yes iff** `AUTH_MODE=bearer`	—	OIDC issuer URL.
`CHARTNAV_JWT_AUDIENCE`	**yes iff** `AUTH_MODE=bearer`	—	Expected `aud` claim.
`CHARTNAV_JWT_JWKS_URL`	**yes iff** `AUTH_MODE=bearer`	—	JWKS endpoint for signing-key lookup.
`CHARTNAV_RUN_SEED`	no	`0`	Entrypoint: run `scripts_seed.py` after migrations. Keep `0` in prod.
`API_HOST` / `API_PORT`	no	`0.0.0.0` / `8000`	Uvicorn bind.

Validation on startup

`app.config._load()` is evaluated at import time. It enforces:

- `CHARTNAV_AUTH_MODE ∈ {header, bearer}` .
- If `AUTH_MODE=bearer` , **all three** JWT vars must be set — otherwise

a `RuntimeError` is raised listing the missing names. The app will

not boot half-configured.

Dev / CI / Prod profiles

Profile	How config is supplied	DB	Auth mode	Seed
Dev	`apps/api/.env.example` → `.env` ; Makefile targets	`sqlite:///apps/api/chartnav.db`	`header`	ad-hoc (`make seed`)
Tests	`apps/api/tests/conftest.py` sets env per-test	per-test SQLite in `tmp_path`	`header`	via `scripts_seed.main()`
CI	`.github/workflows/ci.yml` inline env + services	SQLite + live Postgres service	`header`	seed + re-seed in workflow
Prod	platform env / secret store	`postgresql+psycopg://...` via `infra/docker/docker-compose.prod.yml`	`bearer` (when JWT validation is wired)	off by default

Example files

- `apps/api/.env.example` — full dev contract with comments.
- `infra/docker/docker-compose.prod.yml` — wires the API container to

a Postgres service using these env vars; each has a documented

default so the file is runnable as-is for local parity tests.

Database URL specifics

- **SQLite:** `sqlite:///relative/or/absolute/path.db` (three slashes +

absolute path, or three slashes + relative path).

- **Postgres:** `postgresql+psycopg://user:pass@host:5432/dbname` .

The `+psycopg` driver selector is important — SQLAlchemy defaults to

`psycopg2` which is not installed. `psycopg[binary]` is declared in

`apps/api/pyproject.toml` as the `[postgres]` extra.

Frontend runtime config

The web app reads exactly one env var at build time:

Variable	Required?	Default	Purpose
`VITE_API_URL`	no	`http://localhost:8000`	Base URL the API client points at.

Template: `apps/web/.env.example` . Copy to `.env` to override for a

non-default backend.

Vite substitutes `import.meta.env.VITE_API_URL` into the bundle at build.

Nothing else about the UI comes from env today — dev identity is

selected inside the app and stored in `localStorage`.

What this phase explicitly does NOT do

- No secret store integration (AWS SM, Vault, etc.).
- No `.env` auto-loading in the API (docker-compose and operators load env).
- No per-request feature flags / remote config.

Those are safe to layer on top of this contract without reshaping it.

Deploy Target

Chosen path

Docker image built from `apps/api/Dockerfile`, wired to a Postgres service via `infra/docker/docker-compose.prod.yml`. That's the single supported deploy surface in this phase — no Kubernetes, no Terraform, no cloud-specific glue yet.

Hardened Dockerfile

`apps/api/Dockerfile`:

- `python:3.11-slim` base.
- `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, disabled pip cache.
- Installs `curl` only (for the container healthcheck).
- Installs the API with the `[postgres]` extra (`psycopg[binary]`).
- Copies `app/`, `alembic/`, `alembic.ini`, `scripts/`, `scripts_seed.py`, `entrypoint.sh`.
- Creates a non-root `chartnav` user (uid 1001) and runs as that user.
- `HEALTHCHECK` hits `/health` every 15s with 3 retries.
- `ENTRYPOINT` is `./entrypoint.sh`; `CMD` is the uvicorn invocation.

Entrypoint

`apps/api/entrypoint.sh`:

1. Asserts `DATABASE_URL` is set (fails loudly otherwise).
2. `alembic upgrade head` — migrations always run at start-up.
3. If `CHARTNAV_RUN_SEED=1`, runs `python scripts_seed.py` (idempotent; off by default in prod).
4. `exec "$@"` — hands off to `CMD` so uvicorn becomes PID 1's child.

Migration-on-start is the right default for this app today. It is idempotent, small, and the seed step is gated off in production.

docker-compose.prod.yml

`infra/docker/docker-compose.prod.yml` brings up two services:

- `db` — `postgres:16-alpine` with a persistent volume, healthcheck,

and dev-safe default credentials (operators must override for real

deploys).

- `api` — built from `apps/api/Dockerfile`, depends on `db` with

`condition: service_healthy`, reads all runtime config from env.

All sensitive values (`POSTGRES_PASSWORD`, JWT settings, etc.) are

`${VAR:-default}` so the file is runnable locally for parity tests but

ships nothing secret.

Startup path

```
# clone + cd
cd infra/docker
# optional: override defaults via env
docker compose -f docker-compose.prod.yml up --build
```

Boot sequence per container:

1. Postgres starts, becomes healthy (~5s).

2. API container starts, entrypoint runs migrations.

3. uvicorn binds 0.0.0.0:8000.

4. HEALTHCHECK turns green.

Runtime config path

Every variable is documented in `12-runtime-config.md`. The

image respects only those — there's no bespoke `config.yaml`,

Consul, etc. to reason about.

CI build verification

`.github/workflows/ci.yml` → `docker-build` job:

1. Buildx sets up.

2. Builds `chartnav-api:ci` from `apps/api/`.

3. Runs the container with `DATABASE_URL=sqlite:///./chartnav.db` and

`CHARTNAV_RUN_SEED=1`.

4. Polls `/health`.

5. Runs the same `scripts/smoke.sh` the backend-sqlite job uses.

6. Tears down.

If the image can't boot, serve `/health`, or pass smoke, the job fails.

Out of scope this phase

- Image registry push (no target credentials by default).
- Kubernetes manifests, Helm charts, Terraform.
- TLS termination — expected to be handled by a reverse proxy in front.
- Secret management integration (AWS SM, Vault, etc.).
- Blue/green or rolling deployment orchestration.

Those are the next "hosted deploy" phase's job. The Docker seam is

real and ready to be plugged into any of them.

14 — Postgres parity

Postgres Parity

Why this phase

The app was born on SQLite. This phase proves the same code runs on

Postgres — migrations, seed, auth, scoping, RBAC, state machine.

Code-level changes that made this possible

The DB layer was rewritten to use **SQLAlchemy Core** (`apps/api/app/db.py`).

That single switch buys cross-dialect behavior:

- All SQL uses **named** bind parameters (`:name`). No more `?` .
- Portable functions only — `COALESCE` replaced `IFNULL` ; `CURRENT_TIMESTAMP` already worked on both.
- `insert_returning_id(conn, table, values)` helper encapsulates the

dialect split: `RETURNING id` on Postgres, `cursor.lastrowid` on

SQLite. Callers never touch either.

- `transaction()` context manager gives a SA connection with `begin()`

semantics on either backend.

Consumers changed:

- `apps/api/app/api/routes.py` — all queries now use `:name` binds

and `transaction()` .

- `apps/api/scripts_seed.py` — rewritten to `transaction()` +

`insert_returning_id` , no more raw `sqlite3` .

- `apps/api/app/auth.py` — uses `fetch_one` from the new `db` module.
- `apps/api/alembic/env.py` — honors both `-x sqlalchemy.url=` and

`DATABASE_URL` env, so migrations can run against either backend.

No existing migration (`43ccbf363a8f` , `a1b2c3d4e5f6`) needed changes.

They already used SQLAlchemy constructs and portable defaults.

Proof — `scripts/pg_verify.sh`

Single reproducible command. Launches `postgres:16-alpine` in a

throwaway container, runs migrations + seed, boots the API against it,

runs the full smoke, then does a live **status transition** + reads

the resulting `workflow_events` row back.

Local run on 2026-04-18:

```
==> alembic upgrade head on Postgres
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Running upgrade  -> 43ccbf363a8f, create initial chartnav tables
INFO  [alembic.runtime.migration] Running upgrade 43ccbf363a8f -> a1b2c3d4e5f6, add encounters and wor

==> seed (idempotent, run twice)
Seed complete.
  demo-eye-clinic: organization_id=1 location_id=1
  northside-retina: organization_id=2 location_id=2

==> smoke: http://127.0.0.1:8766
  ok  /health 200
  ok  /health body.status=ok
  ok  /me 401 without auth
  ok  /me 200 role=admin org=1
  ok  /encounters 401 without auth
  ok  /encounters 200 admin1
  ok  /encounters?organization_id=2 (cross-org) -> 403
  ok  GET /encounters/1 as admin1 -> 200
  ok  GET /encounters/3 cross-org -> 404
==> all smoke checks passed
==> extra: status transition on Postgres
  ok  PT-1001 in_progress -> draft_ready (clinician)
  ok  workflow_events row recorded on Postgres
==> Postgres parity: PASS
```

CI coverage

`.github/workflows/ci.yml` → `backend-postgres` job:

1. Spins up a `postgres:16-alpine` service container on port 5432.

2. Installs the API with the `[postgres]` extra.

3. `alembic upgrade head` (direct env `DATABASE_URL=postgresql+psycopg://…`).

4. Seeds twice (idempotency proof).

5. Boots uvicorn, waits for `/health` .

6. Runs `scripts/smoke.sh` .

7. Executes the same `clinician → draft_ready` status transition and

asserts the returned row.

If any step fails, the job fails and nothing merges.

What's covered / not covered

Surface	Verified on Postgres
Schema migrations	✅
Idempotent seed	✅
App boot	✅
`/health`, `/me`	✅
`/encounters` (list + filters + cross-org 403)	✅
`/encounters/{id}` 200 / 404 cross-org	✅
State transition write + event emit	✅
pytest suite (28 tests)	Runs on SQLite only today — see Known Gaps.

The pytest suite still targets SQLite because the TestClient fixture uses `tmp_path` file DBs. Migrating that to a Postgres matrix is the obvious next CI upgrade; the DB layer is ready for it.

15 — Frontend integration

Frontend Integration

Goal

Turn the backend workflow slice into a usable app. An operator should

be able to identify themselves, filter their org's encounters, open one,

see its timeline, append events, and drive role-appropriate status

transitions — all through the UI — without the backend contract changing.

Stack

- Vite 5 + React 18 + TypeScript (already in the repo).
- No component library yet — one small CSS file (`styles.css`) + inline

attributes. Easy to swap later if the UI grows.

- No state library — `useState` / `useEffect` around a typed API module.

Source layout

```
apps/web/src/
├── api.ts            # typed API client (single fetch surface)
├── identity.ts       # dev identity helpers (seeded list + localStorage)
├── App.tsx           # shell, list, detail, timeline, actions
├── main.tsx          # React entrypoint, imports styles.css
├── styles.css        # one CSS file (CSS variables; no build config)
└── vite-env.d.ts     # Vite ambient types
```

`api.ts` — contract

- `API_URL` comes from `VITE_API_URL` (falls back to `http://localhost:8000`).
- Every function takes an `email: string` ; the client sets

`X-User-Email` on each request. When the backend moves to bearer

mode, only this module changes.

- `ApiError(status, errorCode, reason)` — every non-2xx response is

converted to this, so UI code can render the backend's

`{error_code, reason}` envelope directly.

- Exposed functions (match the backend 1:1):
- `getHealth()`
- `getMe(email)`
- `listEncounters(email, filters)`
- `getEncounter(email, id)`
- `getEncounterEvents(email, id)`
- `createEncounterEvent(email, id, {event_type, event_data})`
- `updateEncounterStatus(email, id, status)`
- `listLocations(email)`
- Pure helpers used for UI affordances only:
- `allowedNextStatuses(role, currentStatus)` — mirrors

`authz.TRANSITION_ROLES`. Drives which transition buttons appear.

- `canCreateEvent(role)` — drives the "Add event" composer.
- **Backend remains the source of truth** — the UI never assumes the

server will accept anything; any 4xx surfaces as a banner with the

error code + reason.

Dev identity

- `apps/web/src/identity.ts` ships the seeded demo users:
- Org 1: `admin@`, `clin@`, `rev@chartnav.local`
- Org 2: `admin@`, `clin@northside.local`
- Selector in the header lets operators switch caller without editing

code. "Custom email…" lets a real/unseeded email be entered for

testing 401 paths.

- Selection persists in `localStorage.chartnav.devIdentity`.

UI layers

Header

- Brand wordmark.
- Chip showing the resolved caller (`email · role · org N`) — or the

auth error if `/me` failed.

- Chip with the current API base URL.
- Identity picker.

Layout (two-column, collapses below 920px)

- **Encounter list (left)** — filter bar (`status` , `provider_name` , `location_id`) + row list with patient id, name, provider, and a color-coded status pill.
- **Detail pane (right)** — headline + status pill, facts grid (organization / location / scheduled / started / completed / created), allowed-transition buttons, timeline of events, and "Add event" composer when the current role is permitted to write.

Role-aware actions

- Allowed transitions are shown as buttons. If none are available for the (role, current_status) pair, we show a note instead of fake disabled buttons.
- The "Add event" composer is hidden for reviewers (matches `CAN_CREATE_EVENT`). Reviewers still get a subtle note explaining why.
- After any successful action the UI refreshes: detail, events, and list.
- After a failure, the UI shows the exact `error_code` + `reason` from the backend in a top banner.

UX states

State	Surface
Loading	"Loading…" in list / detail
Empty list	"No encounters match these filters."
401 / unknown	Red chip in the header: `auth: 401 unknown_user — ...`
403 role	Banner: `403 role_cannot_transition — ...`
404 cross-org	Banner: `404 encounter_not_found — ...`
400 bad status	Banner: `400 invalid_transition — ... allowed next ...`

Local dev

```
# one terminal — backend
make install          # venv + pip install -e "apps/api[dev,postgres]"
make reset-db         # fresh SQLite + seed
```

```
make boot             # uvicorn on 8000

# second terminal — frontend
make web-install      # npm install
make web-dev          # vite on 5173

# or both at once (Ctrl-C to stop both)
make dev
```

`VITE_API_URL` in `apps/web/.env.example` defaults to

`http://localhost:8000`, matching `make boot`. Copy to `.env` if you

need to point at a different backend.

Build / verification

```
make web-typecheck    # tsc --noEmit
make web-build        # vite build -> apps/web/dist
```

Verified manually on 2026-04-18:

Check	Result
`tsc --noEmit`	✅
`npm run build` emits dist/	✅ (154 KB JS / 6 KB CSS)
`GET /me` for all 5 seeded users	✅
`GET /encounters` org-scoped (org1 → 2 rows; org2 → `[3]`)	✅
`GET /encounters?status=in_progress`	✅ `['PT-1001']`
`GET /encounters/{id}/events` hydrated	✅ 3 events
Clinician in_progress → draft_ready	✅ 200
Clinician review_needed → completed (denied)	✅ 403 `role_cannot_transition`
Reviewer review_needed → completed	✅ 200
Reviewer POST event (denied)	✅ 403 `role_cannot_create_event`
Admin POST event	✅ 201
Switching identity re-fetches `/me` and list	✅

What this phase explicitly does NOT do

- No real login flow — `X-User-Email` is still dev transport.
- No global state manager (Redux/Zustand) — state lives in `App.tsx`.
- No UI component library.
- No pagination (the backend doesn't paginate yet).

- No optimistic updates — every mutation re-fetches the relevant slice.

Those are future phases. The API client + identity seam make them

cheap to add without rewriting.

Diagram — System architecture

System Architecture

```
flowchart LR subgraph Web["Frontend (apps/web, Vite + React + TS)"] UI["App.tsx
list · detail · timeline · actions"] ApiClient["api.ts
typed client + ApiError"] Ident["identity.ts
localStorage dev caller"] end subgraph Runtime["Backend (apps/api, FastAPI)"] FastAPI["main.py"]
Config["config.py
env-driven settings"] Authn["require_caller
header · bearer"] Authz["authz.py RBAC"] Router["routes.py"] SM["State Machine"] DB["db.py · SA Core"]
Engine[(SA Engine)] end subgraph Storage SQLite[("SQLite
apps/api/chartnav.db")] PG[("Postgres 16")] end subgraph Deploy Dockerfile["Dockerfile
non-root · healthcheck"] Entry["entrypoint.sh
migrate + exec"] Compose["docker-compose.prod.yml"] end subgraph Tooling Alembic["Alembic
DATABASE_URL-aware"] Seed["scripts_seed.py"] PyTest["pytest (28)"] Smoke["scripts/smoke.sh"]
Verify["scripts/verify.sh"] PgVerify["scripts/pg_verify.sh"] DocBuild["scripts/build_docs.py"] Make["Makefile
install · verify · pg-verify · dev · web-*"] CI[".github/workflows/ci.yml"] end UI --> ApiClient ApiClient -->|"HTTP + X-
User-Email"| FastAPI Ident -. stored caller .-> ApiClient FastAPI --> Authn Authn --> Authz Authz --> Router Router
--> SM Router --> DB DB --> Engine Engine --> SQLite Engine --> PG Config -.-> Authn Config -.-> DB Alembic -.-
> Engine Seed -.-> DB PyTest -.-> FastAPI Smoke -.-> FastAPI Verify --> Make PgVerify --> Make DocBuild -->
Make Make --> Alembic Make --> Seed Make --> PyTest Make -. web-dev · web-build .-> UI CI --> Make Dockerfile
--> Entry Entry --> Alembic Entry --> FastAPI Compose --> Dockerfile Compose --> PG
```

Diagram — Encounter status machine

Encounter Status State Machine

```
stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]
```

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer" datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow & CI

UI → Backend request path

sequenceDiagram autonumber participant U as Operator (browser) participant W as apps/web (App.tsx + api.ts) participant Cfg as app.config participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SA Core → SQLite / Postgres U->>W: pick identity · change filter · click transition W->>Authn: fetch with X-User-Email: admin@chartnav.local Cfg-->>Authn: auth_mode=header Authn->>DB: SELECT user WHERE email=:email DB-->>Authn: {role, org} Authn-->>Authz: Caller Authz->>R: Caller R->>DB: scoped SQL (:name binds) DB-->>R: rows R-->>W: 200 + JSON (or 4xx {error_code, reason}) W-->>U: render list · detail · timeline · banner Note over W,R: On 4xx, api.ts wraps into ApiError(status, error_code, reason). App.tsx surfaces it verbatim as a banner.

Role-aware transition flow

sequenceDiagram autonumber participant W as App.tsx participant A as api.allowedNextStatuses() participant B as POST /encounters/{id}/status participant Authz as authz.assert_can_transition W->>A: compute buttons for (role, current) A-->>W: [next1, next2, ...] (UI hint only) W->>B: POST status=next B->>Authz: validate state-machine edge + role edge alt allowed Authz-->>B: ok B-->>W: 200 updated encounter W->>W: refresh detail + events + list; green banner else role forbidden Authz-->>W: 403 role_cannot_transition W->>W: red banner; UI affordances unchanged until next refresh end

CI gate flow

flowchart TD PR[["push / PR"]] --> SQL[backend-sqlite] SQL --> PG[backend-postgres] SQL --> DK[docker-build] SQL --> DOC[docs] subgraph "backend-sqlite" A1[pip install -e .dev,postgres] --> A2[alembic upgrade] A2 --> A3[seed x2] A3 --> A4[pytest 28] A4 --> A5[scripts/verify.sh] end subgraph "backend-postgres (service: postgres:16-alpine)" B1[install] --> B2[alembic upgrade] B2 --> B3[seed x2] B3 --> B4[boot + smoke] B4 --> B5[live status transition] end subgraph "docker-build" D1[buildx build] --> D2[run container] D2 --> D3[smoke] end subgraph "docs" X1[apt chromium] --> X2[build_docs.py] X2 --> X3[upload artifact] end